



Министерство науки и высшего образования Российской Федерации
Федеральное государственное бюджетное образовательное учреждение
высшего образования
«Московский государственный технический университет
имени Н.Э. Баумана
(национальный исследовательский университет)»
(МГТУ им. Н.Э. Баумана)

ФАКУЛЬТЕТ _____ «Информатика и системы управления»

КАФЕДРА _____ «Теоретическая информатика и компьютерные технологии»

Лабораторная работа № 5
«Вычисление собственных значений и собственных векторов
симметричной матрицы методом А.М. Крылова»
по курсу «Численные методы линейной алгебры»

Студент группы ИУ9-71Б Лысенко В. В.

Преподаватель Посевин Д. П.

Москва 2024

1 Цель работы

Реализовать метод вычисления собственных значений и собственных векторов симметричной матрицы методом А.М. Крылова.

2 Задачи

1. Реализовать метод вычисления собственных значений и собственных векторов симметричной матрицы методом А.М. Крылова.
2. Реализовать метод поиска собственных значений действительной симметричной матрицы A размером 4×4 .
3. Проверить решение на матрице приведенной в презентации.
4. Продемонстрировать работу приложения для произвольных симметричных матриц размером $n \times n$ с учетом выполнения пунктов приведенных выше.
5. Сравнить результаты с полученными методом Данилевского.

3 Практическая реализация

Сравнение результатов, полученных методами Данилевского и Крылова, реализовано по следующему принципу:

При сравнении собственных значений вычисляются их разницы с собственными значениями, полученными библиотечным методом. После берётся самая большая разница для каждого из методов.

При сравнении собственных векторов вычисляются углы между векторами, полученными каждым из методов, с векторами, вычисленными библиотечным методом. После берётся самый большой угол для каждого из методов. Углы вычислены в радианах.

Ниже представлен код программы:

```
using Random
using LinearAlgebra
using Plots
using Symbolics
using Polynomials

eps = 1e-10

function generate_symmetric_matrix(n)
    A = rand(Float64, n, n)
    for i in 1:n
        for j in i+1:n
            A[i, j] = A[j, i]
        end
    end
    return A
end

function roots_for_Daniel(coefficients)
    x = -reverse(coefficients)
    append!(x, 1)
    p = Polynomial(x)

    return roots(p)
end

function danilevsky_method(A)
    n = size(A, 1)
    lambda_values = Vector{Float64}()
```

```

lambda_vectors = Vector{Vector{Float64}}{ }()
main_B = I(n)

B = A

for k in 1:n-1
    B = zeros(Float64, n, n)
    for i in 1:n
        B[i, i] = 1.0
    end
    for j in 1:n
        B[n-k, j] = - A[n+1-k, j] / A[n+1-k, n-k]
        if j==n-k
            B[n-k, j] = 1 / A[n+1-k, n-k]
        end
    end
end

A = inv(B) * A * B

main_B *= B
end

lambda_values = roots_for_Daniel(A[1, :])

for val in lambda_values
    my_vector = Vector{Float64}{ }
    for i in 0:n-1
        push!(my_vector, val^i)
    end

    my_vector = main_B * reverse(my_vector)

    push!(lambda_vectors, my_vector / norm(my_vector))
end
return lambda_values, lambda_vectors
end

function roots_for_Krylov(coefficients)
    x = -coefficients
    append!(x, 1)
    p = Polynomial(x)

    return roots(p)
end

```

```

function krylov_method(A)
    n = size(A, 1)
    c = zeros(Float64, n)
    c[1] = 1.0
    c_vectors = Vector{Vector{Float64}}()
    push!(c_vectors, c)
    lambda_vectors = Vector{Vector{Float64}}()

    for i in 1:n
        c = A * c
        push!(c_vectors, c)
    end

    _matrix = zeros(Float64, n, n)

    for i in 1:n
        for j in 1:nc
            _matrix[i, j] = c_vectors[j][i]
        end
    end

    Alu = qrc(_matrix)
    p = Alu \ c_vectors[n+1]

    lambda_values = roots_for_Krylov(p)

    p = reverse(p)
    for val in lambda_values

        betas = Vector{Float64}()
        push!(betas, 1)
        for i in 2:n
            push!(betas, val*betas[i-1] - p[i-1])
        end

        x = reversec(_matrix) * betas
        push!(lambda_vectors, reverse(x / norm(x) * sign(x[1])))
    end

    return lambda_values, lambda_vectors
end

```

```

function check_viete(lambda_values, A)
    trace_A = tr(A)
    det_A = det(A)
    sum_lambda_values = sum(lambda_values)
    product_lambda_values = prod(lambda_values)

    if isapprox(sum_lambda_values, trace_A) && isapprox(product_lambda_values, det_A)
        println("Проверка теоремы Виета пройдена ")
    else
        println("Проверка теоремы Виета НЕ пройдена ")
    end
end

function check_gershgorin(lambda_values, A)
    n = size(A, 1)
    is_valid = true

    for i in 1:n
        row_sum = sum(abs.(A[i, :]) - abs(A[i, i]))
        if abs(lambda_values[i] - A[i, i]) > row_sum
            is_valid = false
        end
    end

    if is_valid
        println("Проверка кругов Гершгорина пройдена ")
    else
        println("Проверка кругов Гершгорина НЕ пройдена ")
    end
end

function are_orthogonal(vectors)
    n = size(vectors, 1)

    is_ortagonal = true
    for i in 1:n
        for j in 1:n
            if i != j
                dot_product = dot(vectors[i], vectors[j])
                if eps > dot_product
                    dot_product = 0
                end
            end
            is_ortagonal = isapprox(dot_product, 0)
        end
    end
end

```

```

        end
        if !is_ortagonal
            break
        end
    end
    if !is_ortagonal
        break
    end
end

if is_ortagonal
    println("Собственные вектора ортогональны ")
else
    println("Собственные вектора не ортогональны ")
end
end

function compare_values(lambda_values_danilevsky, lambda_values_krylov)
    diff = abs.(lambda_values_danilevsky - lambda_values_krylov)
    return maximum(diff)
end

function compare_vectors(lambda_vectors_danilevsky, lambda_vectors_krylov, n)
    angels = Vector{Float64}()
    for i in 1:n
        push!(angels, dot(lambda_vectors_danilevsky[i], lambda_vectors_krylov[i]) / (norm(lambda_vectors_danilevsky[i]) * norm(lambda_vectors_krylov[i])))
    end
    return maximum(angels)
end

A = [2.2 1 0.5 2;
      1 1.3 2 1;
      0.5 2 0.5 1.6;
      2 1 1.6 2]
A = Float64.(copy(A))
println("Симметричная матрица A:")
for i in 1:4
    for j in 1:4
        print(A[i,j], " ")
    end
    println()
end
println()

```

```

lambda_values_A_Krylov, lambda_vectors_A_Krylov = krylov_method(A)
println("Собственные значения по Крылову : ")
println(lambda_values_A_Krylov)
println()
println("Собственные векторы по Крылову : ")
for vec in lambda_vectors_A_Krylov
    println(vec)
end
println()

check_viete(lambda_values_A_Krylov, A)
check_gershgorin(lambda_values_A_Krylov, A)
are_orthogonal(lambda_vectors_A_Krylov)
println()

println("
-----
")

lambda_values_A_Daniel, lambda_vectors_A_Daniel = danilevsky_method(A)
println("Собственные значения по Данилевскому : ")
println(lambda_values_A_Daniel)
println()
println("Собственные векторы по Данилевскому : ")
for vec in lambda_vectors_A_Daniel
    println(vec)
end
println()

check_viete(lambda_values_A_Daniel, A)
check_gershgorin(lambda_values_A_Daniel, A)
are_orthogonal(lambda_vectors_A_Daniel)
println()

compare_values(lambda_values_A_Daniel, lambda_values_A_Krylov)

println("
-----
")
println("
-----
")

B_size = 6
B = generate_symmetric_matrix(B_size)

```



```

println("Симметричная матрица B:")
for i in 1:B_size
    for j in 1:B_size
        print(B[i,j], " ")
    end
    println()
end
println()

lambda_values, lambda_vectors = krylov_method(B)
println("Собственные значения по Крылову :")
println(lambda_values)
println()
println("Собственные векторы по Крылову :")
for vec in lambda_vectors
    println(vec)
end
println()

check_viete(lambda_values, B)
check_gershgorin(lambda_values, B)
are_orthogonal(lambda_vectors)
println()

check_viete(lambda_values, B)
check_gershgorin(lambda_values, B)
are_orthogonal(lambda_vectors)
println()

println("
-----
")

lambda_values, lambda_vectors = danilevsky_method(B)
println("Собственные значения по Данилевскому :")
println(lambda_values)
println()
println("Собственные векторы по Данилевскому :")
for vec in lambda_vectors
    println(vec)
end
println()

check_viete(lambda_values, B)
check_gershgorin(lambda_values, B)
are_orthogonal(lambda_vectors)

```

```

println()

cup = 15

Daniel_values_diffs = Vector{Float64}()
Daniel_vectors_diffs = Vector{Float64}()
Krylov_values_diffs = Vector{Float64}()
Krylov_vectors_diffs = Vector{Float64}()
cups = Vector{Float64}()

for n in 3:cup
    push!(cups, n)
    Array = generate_symmetric_matrix(n)

    classic = eigen(Array)
    classic_values = classic.values
    classic_vectors = Vector{Vector{Float64}}()
    for i in 1:n
        vec = Vector{Float64}()
        for j in 1:n
            push!(vec, classic_vectors[j,i])
        end
        push!(classic_vectors, vec)
    end

    Krylov_values, Krylov_vectors = krylov_method(Array)
    Daniel_values, Daniel_vectors = danilevsky_method(Array)

    push!(Krylov_values_diffs, compare_values(classic_values, Krylov_values))
    push!(Daniel_values_diffs, compare_values(classic_values, Daniel_values))
    push!(Krylov_vectors_diffs, compare_vectors(classic_vectors,
        Krylov_vectors, n))
    push!(Daniel_vectors_diffs, compare_vectors(classic_vectors,
        Daniel_vectors, n))
end

println()

p1 = plot(cups, Krylov_values_diffs, label="Krylov values", xlabel="Matrix
size", ylabel="max error")
plot!(cups, Daniel_values_diffs, label="Daniel values", color=:red)
xticks!(0:1:15)
yticks!(0:0.01:1)

```

```

display(p1)

p3 = plot(cups, Krylov_vectors_diffs, label="Krylov vectors", xlabel="Matrix
      size", ylabel="angle error")
plot!(cups, Daniel_vectors_diffs, label="Daniel vectors", color=:red)
xticks!(0:1:15)
display(p3)

```

4 Результаты

Симметричная матрица A:

```

2.2  1.0  0.5  2.0
1.0  1.3  2.0  1.0
0.5  2.0  0.5  1.6
2.0  1.0  1.6  2.0

```

Собственные значения по Крылову:

```

[-1.4200865939506184, 0.22263592713261193, 1.5454183350534119, 5.652032331764598]

```

Собственные векторы по Крылову:

```

[-0.2220428365454733, 0.5159103236551116, -0.7572742312071337, 0.3332705439047424]
[-0.5219205710113863, -0.454869321614003, 0.1534470183752579, 0.7050863992173643]
[0.6289297646710822, -0.5725742255591175, -0.4856537967630995, 0.20185761572390565]
[0.5317360693095475, 0.4461941219086931, 0.40881553418500766, 0.592484107110384]

```

Проверка теоремы Виета пройдена

Проверка кругов Гершгорина НЕ пройдена

Собственные вектора ортогональны

Собственные значения по Данилевскому:

```

[-1.4200865939506204, 0.22263592713261518, 1.5454183350534154, 5.652032331764597]

```

Собственные векторы по Данилевскому:

```

[-0.2220428365454715, 0.5159103236551118, -0.7572742312071336, 0.3332705439047436]
[-0.5219205710113893, -0.45486932161400206, 0.15344701837525604, 0.705086399217363]
[0.62892976467108, -0.5725742255591186, -0.4856537967631012, 0.20185761572390468]
[0.5317360693095103, 0.4461941219087124, 0.4088155341850386, 0.5924841071103819]

```

Проверка теоремы Виета пройдена

Проверка кругов Гершгорина НЕ пройдена

Собственные вектора ортогональны

Симметричная матрица B:

```
0.9682797322868534 0.2877789941354729 0.6562910904520035 0.805321033656521 0.7657979940071944 0.030179453092668118
0.2877789941354729 0.11509296849926431 0.9297506542185364 0.9570251838915355 0.837526898433673 0.9104889040938005
0.6562910904520035 0.9297506542185364 0.9458893055696275 0.8299714116860953 0.8146456844328853 0.3566788915999115
0.805321033656521 0.9570251838915355 0.8299714116860953 0.3076042483328504 0.8366956668398332 0.14119084738655974
0.7657979940071944 0.837526898433673 0.8146456844328853 0.8366956668398332 0.5630320466558651 0.5969853519005602
0.030179453092668118 0.9104889040938005 0.3566788915999115 0.14119084738655974 0.5969853519005602 0.31785709843359966
```

Собственные значения по Крылову:

```
[-1.0878276553148971, -0.40789225703881904, -0.23629735559250722, 0.1533060473661121, 0.8720163099496578, 3.9244503104110207]
```

Собственные векторы по Крылову:

```
[-0.12170193905864155, -0.7509899872475708, 0.11934727253183645, 0.4820979440680972, -0.016736828875418233, 0.41744503411464895]
[0.22951520488534172, 0.3212226501806564, -0.12854951815091822, 0.35220594321021365, -0.8029376995574952, 0.24260219561505145]
[0.37703138027661964, -0.18893369705713065, 0.4215471651186435, -0.6620991747173867, -0.2030568776685491, 0.4060073536938978]
[0.37034806307705326, 0.0030552702989861324, -0.7448792290973417, -0.09047002886338022, 0.3207361876431599, 0.4437695160456788]
[-0.7114476907356906, 0.35602956427353616, 0.03528963437206183, -0.13978276383174348, 0.04928307675841649, 0.5864057739285301]
[0.3834621787943849, 0.4127562487355606, 0.4852203759841448, 0.4212281822654889, 0.45659425084489236, 0.24746460727651537]
```

Проверка теоремы Виета пройдена

Проверка кругов Гершгорина НЕ пройдена

Собственные вектора ортогональны

Проверка теоремы Виета пройдена

Проверка кругов Гершгорина НЕ пройдена

Собственные вектора ортогональны

Собственные значения по Данилевскому:

```
[-1.0878276553144233, -0.40789225703978493, -0.23629735559437065, 0.15330604736602838, 0.8720163099495899, 3.924450310411023]
```

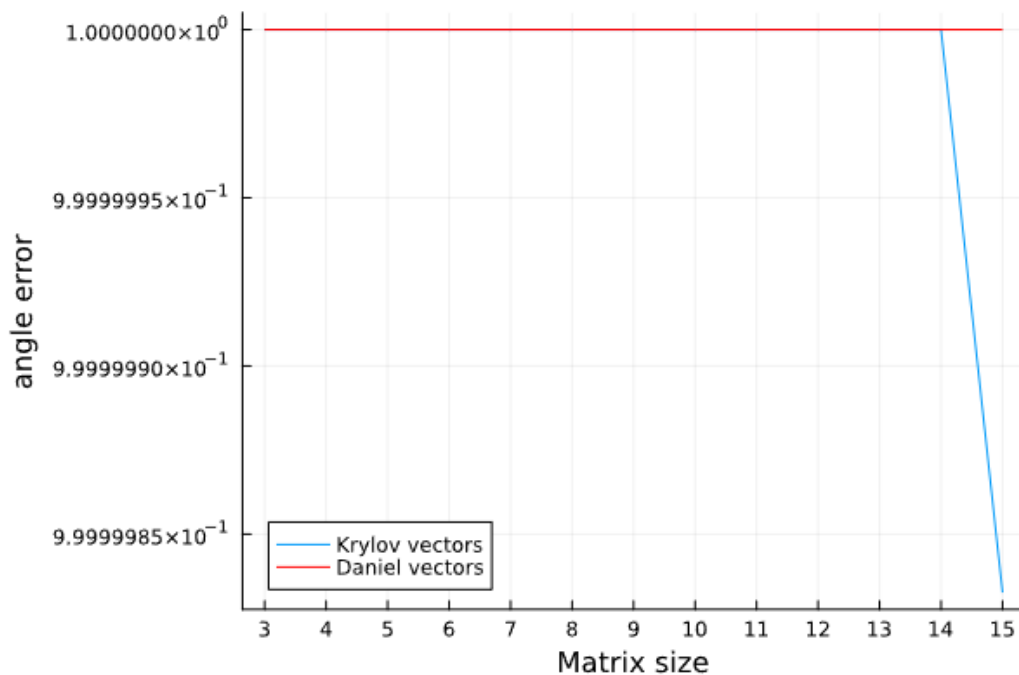
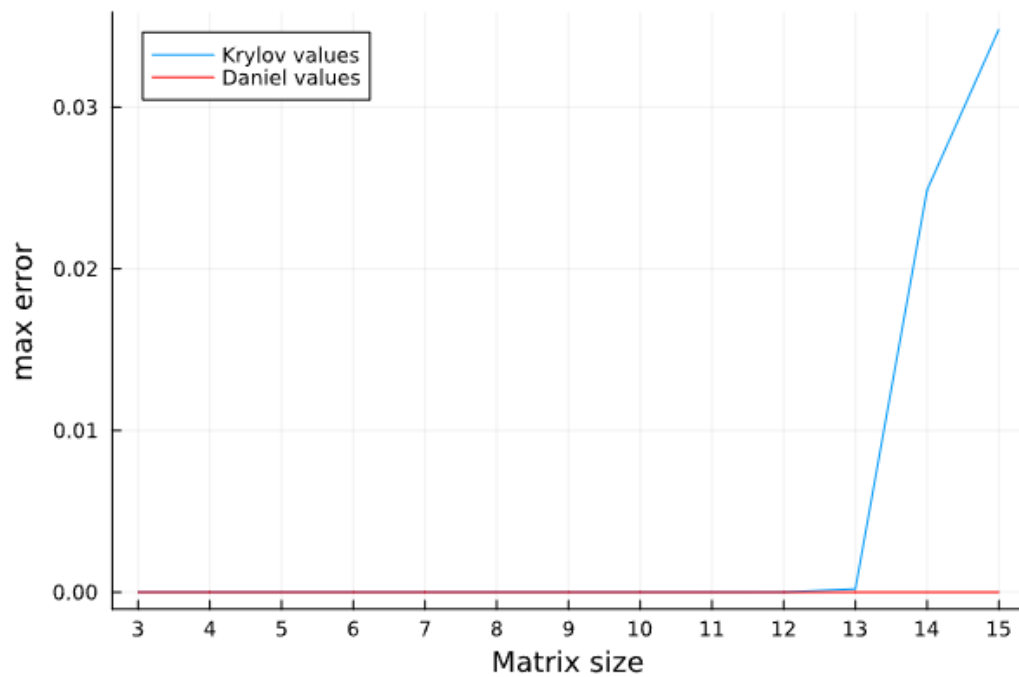
Собственные векторы по Данилевскому:

```
[-0.12170193905881853, -0.7509899872474459, 0.11934727253181483, 0.48209794406820006, -0.016736828875336653, 0.41744503411471273]
[0.22951520487815077, 0.32122265017931645, -0.12854951815500748, 0.35220594322212123, -0.8029376995541702, 0.24260219561517868]
[0.37703138027979927, -0.18893369705350613, 0.421547165117071, -0.6620991747180193, -0.20305687767320432, 0.4060073536909047]
[0.3703480630764977, 0.0030552702983289923, -0.7448792290961029, -0.09047002886427204, 0.32073618764350836, 0.4437695160477928]
[-0.7114476907356775, 0.35602956427349164, 0.0352896343721334, -0.1397827638317601, 0.049283076758442255, 0.5864057739285626]
[0.38346217879445743, 0.41275624873644784, 0.4852203759822466, 0.42122818226810954, 0.4565942508436489, 0.2474646072764788]
```

Проверка теоремы Виета пройдена

Проверка кругов Гершгорина НЕ пройдена

Собственные вектора ортогональны



5 Выводы

В ходе выполнения лабораторной работы был реализован метод А.М. Крылова по вычислению собственных значений и собственных векторов. Метод был реализован для симметричных матриц произвольного размера. Корректность всех вычислений была проверена на матрице, приведённой в презентации. Также результаты вычислений были сравнены с результатами метода Данилев-

ского. Из приведённых графиков можно заключить, что метод Данилевского более точно вычисляет собственные значения матрицы, в то время как метод Крылова более точен в вычислении собственных векторов.